

TRABAJANDO EL BACKEND

1. Creamos una carpeta llamada backend en nuestro proyecto



2. Ingresamos a la terminal con `cd` y el nombre de nuestra carpeta, inicializamos nuestro proyecto con `npm init -y`
3. Instalamos las dependencias que vamos a trabajar en este caso vamos a trabajar con node express y mongo y cors

- Instalamos :node express : `npm install express`
- Instalamos: `npm install mongoose`
- Instalamos `npm install cors`
- Instalamos `npm install -g nodemon`

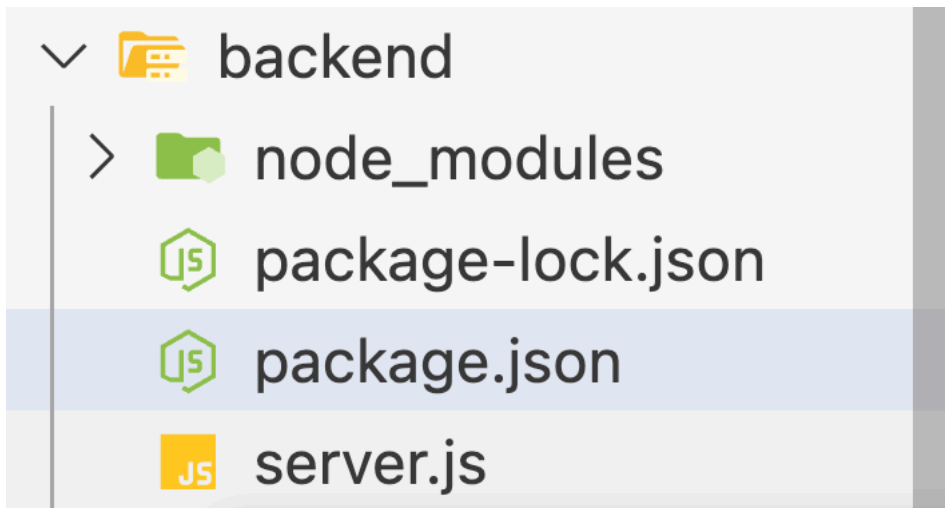
4. Ingresamos al json y configuramos lo siguiente: para levantar nuestros servidores

```
"start": "node server.js",  
"dev": "nodemon server.js"
```



```
"type": "module",
```

5. Creamos el server.js



6. En nuestro servidor escribimos la siguiente estructura que nos permite conectarme con el backend

```
import express from 'express';
import cors from 'cors';

const app = express();

// Habilitar CORS para todas las rutas
app.use(cors());

app.get('/', (req, res) => {
  res.send('Servidor con CORS activado 🚀');
});

app.listen(8081, () => console.log('Servidor corriendo en http://localhost:8081'));
```

7. Creamos la conexión a la base de datos en mongo Atlas,

- Creamos un usuario en Mongo Atlas , creamos el clúster la base de datos y la colección, creamos el usuario y la clave que son fundamentales en la conexión de la base de datos

2. Install your driver

Run the following on the command line

```
npm install mongodb
```

[View MongoDB Node.js Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

☐ View full code sample

```
mongodb+srv://adsotarde:<db_password>@adsotarde.ech1xrh.mongodb.net/?  
appName=ADSOTARDE
```

Replace **<db_password>** with the password for the **adsotarde** database user. Ensure any option params are [URL encoded](#).

RESOURCES

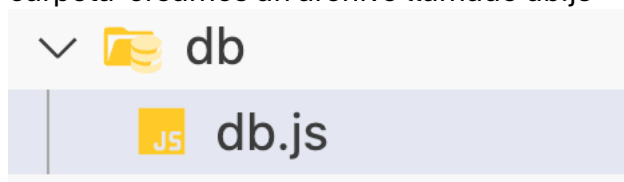
[Get started with the Node.js Driver](#)

[Node.js Starter Sample App](#)

[Access your Database Users](#)

[Troubleshoot Connections](#)

8. Creamos en la el backend una carpeta de que va llamar db y dentro de esa carpeta creamos un archivo llamado db.js



9. Creamos la conexión a nuestra base de datos con el siguiente código

```
import mongoose from "mongoose";  
  
const uri = "mongodb+srv://adsotarde:adsotarde@adsotarde.ech1xrh.mongodb.net/ECOMMER?retryWrites=true&w=majority";  
  
mongoose.connect(uri)  
  .then(() => console.log("conectado a la base de datos"))  
  .catch(err => console.error("error de al conectar a la base datos",err));
```

10. Para realizar las pruebas si ya quedo conectado nuestra base de datos vamos a nuestro server y configuramos la ruta.

```
import "../db/db.js"
```

11. Levantamos el servidor con npm run dev

```
[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Servidor corriendo en http://localhost:8081
conectado a la base de datos
```

De esa manera validamos la conexión a nuestra base de datos.

12. Vamos a crear las siguientes carpetas



- Un **modelo** (model) en Mongoose es como **un plano o molde** que define qué campos y qué tipo de datos tendrá una colección en MongoDB.
- La carpeta **routes** (también llamada **router**) se usa en el backend para **organizar las rutas o endpoints** de tu aplicación.
En otras palabras, ahí defines **qué hace tu backend cuando recibe una petición HTTP** (por ejemplo: GET, POST, PUT, DELETE).
- Un **endpoint** (punto final) es una **dirección URL específica del backend** a la que el **frontend o Postman** puede enviar una petición para hacer algo, como:
 - Registrar un usuario
 - Consultar productos
 - Eliminar un pedido
 - Actualizar un perfil

13. Dentro de la carpeta Models vamos a crear un archivo llamado Productos.js

```
1 import mongoose from "mongoose";
2 const productSchema = new mongoose.Schema({
3   productId: { type: String, required: true, unique: true },
4   Nombre: { type: String, required: true },
5   Descripcion: { type: String, required: true },
6   Precio: { type: Number, required: true },
7   Imagen : { type: String, required: true }
8 });
9
10
11 // Forzamos que se ingrese a la coleccion que esta creada
12 const Product = mongoose.model("Productos", productSchema, "Productos");
13
14 export default Product;
```

Creamos una carpeta de routes y agregamos la ruta

```
import express from "express";
import Productos from "../models/Productos.js";

const router = express.Router();
router.post("/", async function (req, res) {
  try {
    const { productId, Nombre, Descripcion, Precio, Imagen } = req.body;
    const newProduct = new Productos({
      productId,
      Nombre,
      Descripcion,
      Precio,
      Imagen
    });
    await newProduct.save();
    res.status(201).json({ message: " producto Ingresado" });
  } catch (error) {
    console.error(" Error al guardar el producto:", error);
    res.status(400).json({
      message: "Error al ingresar el producto",
      error: error.message
    });
  }
});

export default router;
```

Qué significa `async` y `await`?

- `async` = función asíncrona

- `await` = **esperar a que algo termine**

Estas dos palabras se usan juntas para **manejar procesos que tardan un poco**, como:

- Consultar una base de datos
- Hacer una petición a un servidor
- Leer un archivo
- El bloque `try` contiene el código que **puede fallar**.
- El bloque `catch` se ejecuta **si ocurre un error**, y te permite **evitar que el programa se detenga**.

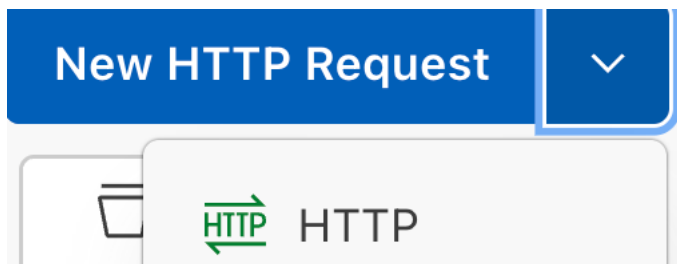
Vamos al server importamos las rutas que estamos trabajando

```
import ProductosRoute from "../routes/Productos.js"
import UsersRoutes from '../routes/Users.js';
import ConsultarRoutes from '../routes/Consultar.js';
```

Después de la ruta Bienvenidos colocamos las rutas para poder que funcione todo.

```
app.use("/api/productos", ProductosRoute);
app.use("/api/users", UsersRoutes);
app.use("/api", ConsultarRoutes);
```

Para verificar que los datos se estén guardando en la base de datos, utilizaremos la extensión de visual estudio code Portman



Seleccionamos el método Post

GET Enter URL or paste text Send

Params Authorization Headers (7) Body Scripts Settings Code Cookies

Query Params

Key	Value
Key	Value

Pegamos la ruta

`http://localhost:8081/api/productos`

Seleccionamos body, raw y que sea formato json

Params Authorization Headers (8) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

1

Ahora agregamos el json para hacer la prueba

```
1 {
2   "productId": "PROD001",
3   "Nombre": "Laptop Lenovo ThinkPad",
4   "Descripcion": "Computador portátil con procesador Intel i7, 16GB RAM y 512GB SSD.",
5   "Precio": 3500000,
6   "Imagen": "https://example.com/images/laptop-lenovo.jpg"
7 }
8
```

dy Cookies Headers (8) Test Results Status: 201 Created Time: 137 ms Size: 305 B

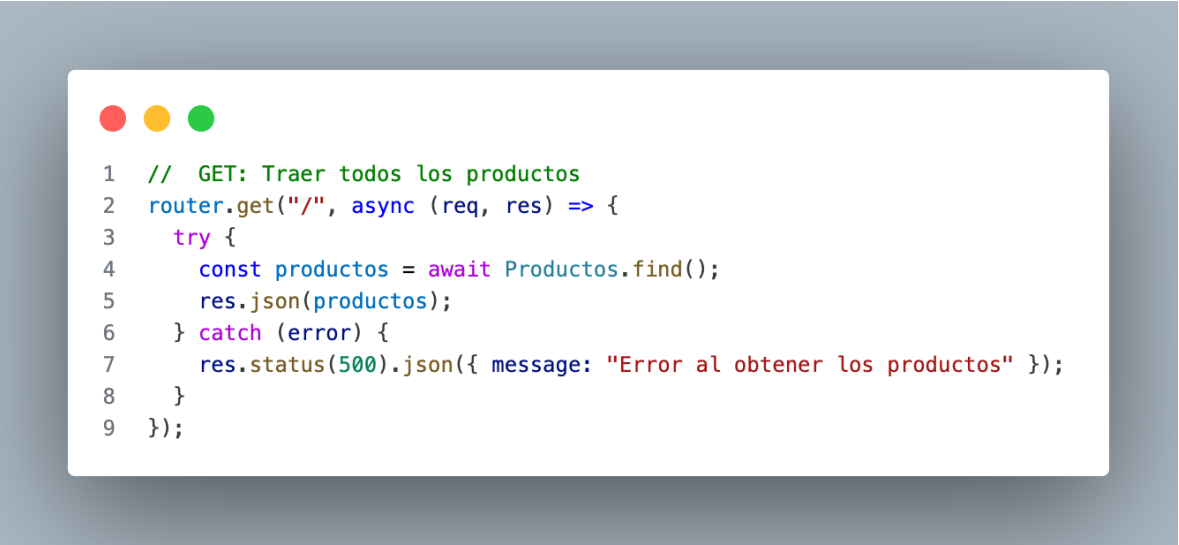
Pretty Raw Preview JSON

```
1 {"message": " producto Ingresado"}
```

Validamos la base de datos y debe quedar creado nuestro producto

```
_id: ObjectId('690a7e84a295678297190cb9')
productId: "Macbook pro m7"
Nombre: "Mac pro m5 de 16 Pulgadas"
Descripcion: "Chip M5, 16 GB de RAM, 512 Ssd"
Precio: 1200
Imagen: "https://unsplash.com/es/fotos/computadora-portatil"
__v: 0
```

Trabajaremos el método get para traer productos



```
1 // GET: Traer todos los productos
2 router.get("/", async (req, res) => {
3   try {
4     const productos = await Productos.find();
5     res.json(productos);
6   } catch (error) {
7     res.status(500).json({ message: "Error al obtener los productos" });
8   }
9 });
```

Seguimos con el usuario



```
1 import mongoose from "mongoose";
2
3 const UsersSchema = new mongoose.Schema({
4   Nombre: { type: String, required: true },
5   Apellido: { type: String, required: true },
6   Telefono: { type: Number, required: true },
7   Correo_Electronico: { type: String, required: true },
8   passwords: { type: String, required: true, minlength: 8 }
9 });
10
11 // Forzamos que se use la colección "User"
12 const Users = mongoose.model("Users", UsersSchema, "Users");
13
14 export default Users;
15
```

Creamos la ruta



```
1 import express from "express";
2 import Users from "../models/Users.js";
3
4 const router = express.Router();
5
6 // Ruta para registrar usuario
7 router.post("/register", async (req, res) => {
8   try {
9     const { Nombre, Apellido, Telefono, Correo_Electronico, passwords } = req.body;
10
11     // Validar que no falte ningún campo
12     if (!Nombre || !Apellido || !Telefono || !Correo_Electronico || !passwords) {
13       return res.status(400).json({ message: "Todos los campos son obligatorios." });
14     }
15   }
16 }
```

Verificar si el usuario existe

```
1 // Verificar si ya existe el correo
2   const existeUsuario = await Users.findOne({ Correo_Electronico });
3   if (existeUsuario) {
4     return res.status(400).json({ message: "El correo ya está registrado." });
5   }
```

Cremos y guardamos el usuario de la base de datos

```
1 // Crear y guardar usuario
2   const nuevoUsuario = new Users({ Nombre, Apellido, Telefono, Correo_Electronico, passwords });
3   await nuevoUsuario.save();
4
5   res.status(201).json({ message: "Usuario registrado exitosamente." });
6 } catch (error) {
7   res.status(500).json({ message: "Error al registrar el usuario.", error: error.message });
8 }
9 });
10
11 export default router;
```

Hacemos lo mismo con potman para hacer las pruebas

Encriptamos la contraseña

Instalamos la dependencia

```
npm install bcrypt
```

Importamos la Libreria en Users.js



```
1 import bcrypt from "bcrypt";
```

Creamos la constante antes de Guardar el usuario



```
1 const saltRounds = 10;  
2 const hashedPassword = await bcrypt.hash(passwords, saltRounds)
```

Configuramos la password

```

1 const nuevoUsuario = new Users({ Nombre, Apellido, Telefono, Correo_Electronico, passwords:hashedPassword });
2   await nuevoUsuario.save();

```

Hacemos la prueba y ahora nos debe crear la passwords encriptada

Creamos el json para hacer la prueba con potman

```

1  {
2    "Apellido": "Pérez",
3    "Telefono": "3001234567",
4    "Correo_Electronico": "juan.perez@example.com",
5    "passwords": "miContraseña123"
6  }

```

Verificamos y debe quedar la siguiente manera

```

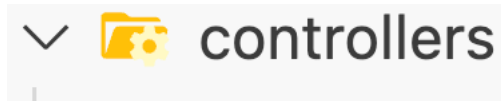
_id: ObjectId('69129fd8c7f5d48f3eb77f8d')
Nombre : "Juan"
Apellido : "Pérez"
Telefono : 3001234567
Correo_Electronico : "juan.perez@example.com"
passwords : "$2b$10$0KiLkAskPZVBnD0jWPxG5.HQCE9KFxv7LAr0H59TZ5HOWEDl..."
__v : 0

```

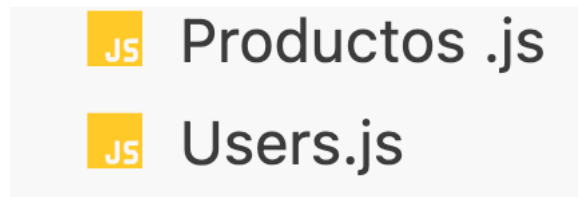


Estructura de node js express

1. Creamos la carpeta de controllers



2. Creanos los controllers



Dentro de Productos js traeremos la lógica de las rutas

Creamos la siguiente estructura

```
1 import Productos from "../models/Productos.js";  
2  
3 // Crear un nuevo producto  
4 export const crearProducto = async (req, res) => {
```

Y copiamos desde try hasta el final



```
1  try {
2    const { productId, Nombre, Descripcion, Precio, Imagen } = req.body;
3
4    const newProduct = new Productos({
5      productId,
6      Nombre,
7      Descripcion,
8      Precio,
9      Imagen
10   });
11
12   await newProduct.save();
13   res.status(201).json({ message: "Producto ingresado exitosamente" });
14 } catch (error) {
15   console.error("Error al guardar el producto:", error);
16   res.status(400).json({
17     message: "Error al ingresar el producto",
18     error: error.message
19   });
20 }
21 };
22
23 // Obtener todos los productos
24 export const obtenerProductos = async (req, res) => {
25   try {
26     const productos = await Productos.find();
27     res.json(productos);
28   } catch (error) {
29     console.error("Error al obtener productos:", error);
30     res.status(500).json({
31       message: "Error al obtener los productos",
32       error: error.message
33     });
34   }
35 };
36
```

Ahora en la ruta hacemos lo siguiente



```
1 import express from "express";
2 import { crearProducto, obtenerProductos } from "../controllers/Productos .js";
3
4 const router = express.Router();
5
6 // Ruta para crear un producto
7 router.post("/", crearProducto);
8
9 // Ruta para obtener todos los productos
10 router.get("/", obtenerProductos);
11
12 export default router;
13
```

Así se hacemos con User

Creamos el user.js en controllers



```
1 import bcrypt from "bcrypt";
2 import Users from "../models/Users.js";
3
4 export const registrarUsuario= async(req,res)=>{
```

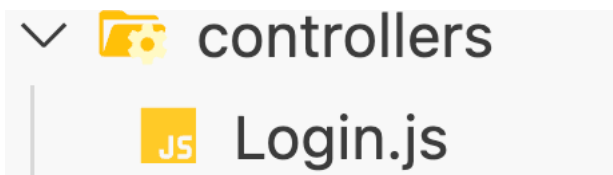
Ahí copiamos desde el try

```
1  try {
2    const { Nombre, Apellido, Telefono, Correo_Electronico, passwords } = req.body;
3
4    // Validar que no falte ningún campo
5    if (!Nombre || !Apellido || !Telefono || !Correo_Electronico || !passwords) {
6      return res.status(400).json({ message: "Todos los campos son obligatorios." });
7    }
8
9    // Verificar si ya existe el correo
10   const existeUsuario = await Users.findOne({ Correo_Electronico });
11   if (existeUsuario) {
12     return res.status(400).json({ message: "El correo ya está registrado." });
13   }
14
15   // Encriptar contraseña
16   const saltRounds = 10;
17   const hashedPassword = await bcrypt.hash(passwords, saltRounds);
18
19   // Crear y guardar usuario
20   const nuevoUsuario = new Users({
21     Nombre,
22     Apellido,
23     Telefono,
24     Correo_Electronico,
25     passwords: hashedPassword
26   });
27
28   await nuevoUsuario.save();
29   res.status(201).json({ message: "Usuario registrado exitosamente." });
30
31 } catch (error) {
32   res.status(500).json({
33     message: "Error al registrar el usuario.",
34     error: error.message
35   });
36 }
37
38 };
39
```

Y en rutas queda de esta manera

```
1 import express from "express";
2 import { registrarUsuario } from "../controllers/Users.js";
3
4 const router = express.Router();
5
6 // Ruta para registrar usuario
7 router.post("/register", registrarUsuario);
8
9 export default router;
10
```

VAMOS A CREAR CONTROLES DE login



Creamos los controllles



```

1  import bcrypt from "bcrypt";
2  import Users from "../models/Users.js";
3
4  export const LoginUsuario=async(req,res)=>{
5      try {
6          const {Correo_Electronico,passwords}=req.body;
7
8          //validamos que los campos esten presentes
9          if (!Correo_Electronico||!passwords){
10              return res.status(400).json({message:" Correo y contraseña son obligatorios"});
11          }
12          //buscamos el el usuario bd
13          const usuario= await Users.findOne({Correo_Electronico});
14          if(!usuario){
15              return res.status(404).json({message:" Usuario no encontrado"});
16          }
17
18          // comparar las contraseñas scriptada en la bd
19          const passwordsValida= await bcrypt.compare(passwords,usuario.passwords);
20          if(!passwordsValida){
21              return res.status(401).json({message:" Contraseña incorrecta"});
22          }
23          // validar inicio exitoso
24          res.status(200).json({message:" Inicio de sesion correcto"});
25      } catch (error) {
26          res.status(500).json({message:" error al iniciar sesion",error:error.message});
27      }
28  };

```

Realizamos la estructura de nuestra ruta



```

1  import express from "express";
2  import { LoginUsuario } from "../controllers/Login.js";
3
4  const router = express.Router();
5
6  router.post("/",LoginUsuario);
7
8  export default router;

```

Configuramos el Server



```
1 import LoginUsuario from './routes/Login.js';
```



```
1 app.use("/api/login",LoginUsuario);
```

Realizamos pruebas y tiene que quedar todo ok

Conexión del backend y fronted

Vamos a conectar el backend con el fronted

1. Vamos a la página de productos y borramos y la dejamos de la siguiente manera



```
1 <!-- GRID DE PRODUCTOS -->
2 <div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-6" id="products-grid">
```

Aquí es importante tener el id ya que es donde vamos a trabajar con JavaScript

2. Vamos a crear un javascript en el frontend

 **productos.js**

3. Desde ahí vamos a crear nuestro código el cual nos va a conectar con el frontend

```
1
2 // Función para cargar productos
3 async function cargarProductos() {
4   try {
5     const response = await fetch('http://localhost:8081/api/productos');
6     const productos = await response.json();
```

- Vamos a crear una función cargar productos
- Try catch para verificar errores
- Se crea una constante de response para que nos traiga la respuesta del servidor
- `fetch()` = función que realiza la conexión HTTP (por ejemplo GET, POST, etc.)
- Creamos una constante para productos que va a venir con json y los convierte en un objeto de javascript

4. vamos a capturar los productos en el grid de la tarjeta que tenemos

```
1 const grid = document.getElementById('products-grid');
2 grid.innerHTML = productos.map(producto => `
```


Busca en tu HTML el elemento con ese `id`. Ese será el contenedor donde vas a mostrar todos los productos.

Inserta todo ese contenido dentro del `<div id="products-grid">`.

```
1 <div class="bg-white rounded-xl shadow-lg overflow-hidden hover:shadow-2xl transition duration-300 transform hover:-translate-y-1 product-card"
2   data-category="laptops"
3   data-price="{{producto.Precio}}"
4   data-product-id="{{producto.productId}}">
```

```
1 <!-- Imagen del producto -->
2 <div class="bg-linear-to-br from-gray-100 to-gray-200 h-48 flex items-center justify-center relative overflow-hidden">
3   
7
8 <!-- Etiqueta de descuento -->
9 <div class="absolute top-3 right-3 bg-red-500 text-white px-2 py-1 rounded-full text-xs font-bold">
10   -15%
11 </div>
12 </div>
```

```
1 <!-- Etiqueta de descuento -->
2 <div class="absolute top-3 right-3 bg-red-500 text-white px-2 py-1 rounded-full text-xs font-bold">
3   -15%
4 </div>
5 </div>
6
```



```
1  <!-- Contenido del producto -->
2  <div class="p-6">
3    <!-- Título -->
4    <h3 class="text-lg font-bold mb-2 text-gray-800">
5      ${producto.Nombre}
6    </h3>
7
8    <!-- Descripción -->
9    <p class="text-sm text-gray-600 mb-4">
10     ${producto.Descripcion}
11  </p>
```



```
1  <!-- Precio y calificación -->
2  <div class="flex items-center justify-between mb-4">
3    <div>
4      <span class="text-2xl font-bold text-blue-600">
5        ${producto.Precio || 0}.toLocaleString('es-CO')}
6      </span>
7    </div>
8    <div class="flex text-yellow-400">
9      *****
10    </div>
11  </div>
```



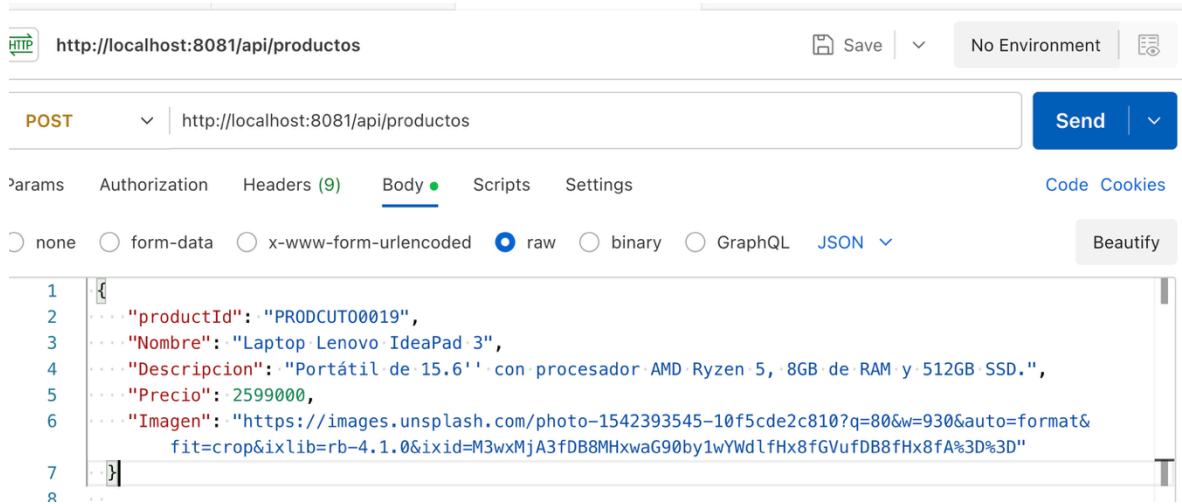
```
1  `).join('');
2      console.log('Productos cargados correctamente');
3  } catch (error) {
4      console.error('Error al cargar productos:', error);
5  }
6  }
7
8  // Llamar la función cuando cargue la página
9  cargarProductos();
10
11  setInterval(() => {
12      cargarProductos();
13  }, 5000); // 5000 ms = 5 segundos
14
```



```
1  <script src="./java/productos.js"></script>
```

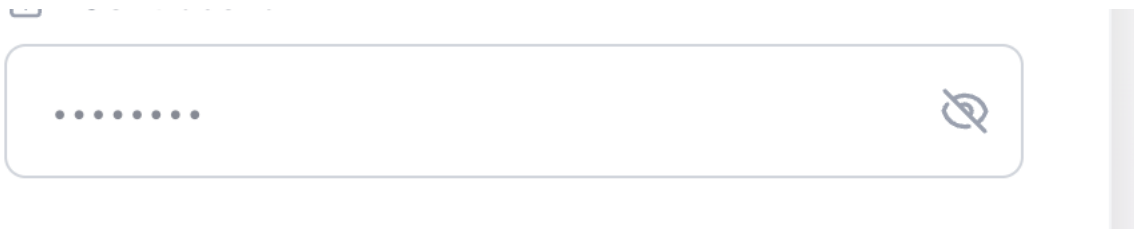
Borramos los productos que tengamos en la base de datos

Realizamos la prueba con Postman



Y apenas le damos send, automáticamente nos debe salir los productos en el frontend

3. Seguimos con la estructura de ocultar y abrir contraseña



- Creamos un javascript que se llame toggle.js



- Creamos la logica para que cuando el damos click se nos abra el ojito y así sucesivamente

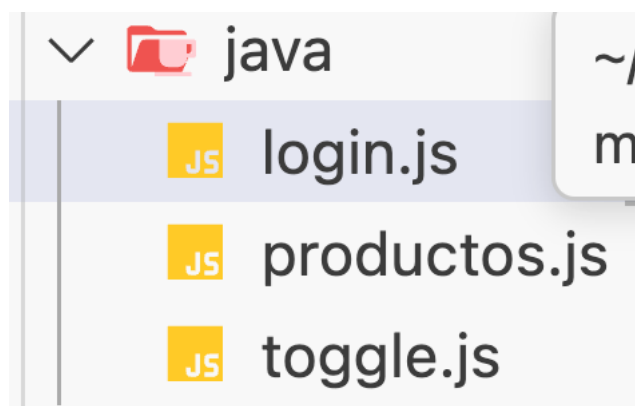
```

1  // visibilidad al ojito
2  document.getElementById('toggle-password').addEventListener('click', function () {
3
4      const passwordInput = document.getElementById('password');
5      const eyeOpen = document.getElementById('eye-icon-open');
6      const eyeClosed = document.getElementById('eye-icon-closed');
7
8      // Verificamos si la contraseña está oculta
9      const isHidden = passwordInput.type === 'password';
10
11     // Cambiamos entre password <-> text
12     passwordInput.type = isHidden ? 'text' : 'password';
13
14     // Alternamos iconos según el estado
15     eyeOpen.classList.toggle('hidden', !isHidden);
16     eyeClosed.classList.toggle('hidden', isHidden);
17 });
18 // Toggle menú móvil
19 document.getElementById('mobile-menu-button').addEventListener('click', () => {
20     document.getElementById('mobile-menu').classList.toggle('hidden');
21 });
22
23

```

Por último, llamamos al Login el js para que nos funcione

4. Vamos a crear nuestro js de login para que nos valide la información con el backend
- Vamos a la carpeta de java y creamos un js de que se llame Login



- Comenzamos a crear la lógica del js



```
1 // Script de Login - TechStore pro
2
3 // Verifica si la pagina esta cargada
4
5 document.addEventListener('DOMContentLoaded',function(){
6     console.log('✅ Pagina Cargada correcta - Sistema listo ');
```

2. creamos la constante del api



```
1 const API_URL = "http://localhost:8081/api/login";
2
```

Preparamos el formulario del login



```

1  // Enviar formulario del login
2
3  document.getElementById('login-form').addEventListener('submit', async function(e){
4      e.preventDefault();
5
6      // preparamos los elementos de la pagina
7      const btn=document.getElementById('login-btn');
8      const errorDiv=document.getElementById('login-error');
9      const errorMsg=document.getElementById('login-error-message');
10
11
12      errorDiv.classList.add('hidden')

```

Recoger todos los campos que vamos a trabajar



```

1  // recoger todos los datos
2      const datos ={
3          Correo_Electronico: document.getElementById('email').value.trim(),
4          passwords:document.getElementById('password').value
5      };
6
7      // Validamos lo campos
8
9      if (!datos.Correo_Electronico || !datos.passwords){
10         errorMsg.textContent='Por favor completa todos los datos ';
11         errorDiv.classList.remove('hidden');
12         return;
13     }

```



```

1  // cambio de boton mientras procesa
2
3      btn.disabled=true;
4      btn.textContent='Iniciando sesion...'

```



```
1 // envia los datos del servidor
2     try {
3         const response= await fetch(API_URL,{
4             method:'POST',
5             headers:{'Content-Type': 'application/json'},
6             body: JSON.stringify(datos )
7         });
8
```



```
1 // RECIBIMOS RESPUESTA DEL SERVIDOR
2     const result = await response.json();
3     if(response.ok){
4
5         console.log('201- Inicio de sesion');
6         // guardamos sesion completa
7
8         localStorage.setItem("sesionActiva",true);
9         localStorage.setItem("usuario",JSON.stringify({
10             id: result.usuario.id,
11             nombre: result.usuario.nombre,
12             apellido: result.usuario.apellido,
13             email: result.usuario.email,
14             telefono: result.usuario.telefono
15
16             })));
```




```

1 // mensaje de exito
2     errorDiv.className='bg-green-50 border-green-200 text-green-800 px-4 py-3 rounded-lg';
3     errorMsg.textContent='Inicio de sesion exitoso';
4     errorDiv.classList.remove('hidden');
5
6     // redirigir a porductos
7
8     setTimeout(() => window.location.href = 'productos.html', 1500);

```



```

1
2     // credenciales son validas
3 } else{
4     // credenciales son validas
5     errorMsg.textContent=result.message || ' Credenciales incorrectas';
6     errorDiv.classList.remove('hidden');
7     btn.disabled = false;
8     btn.innerHTML = 'Iniciar Sesión';
9 }
10 } catch (error) {
11     console.error('Error 404 - Error de conexión con el servidor');
12     errorMsg.textContent = 'Error de conexión con el servidor';
13     errorDiv.classList.remove('hidden');
14     btn.disabled = false;
15     btn.innerHTML = 'Iniciar Sesión';
16 }
17 }
18
19 });
20 });

```

Listo antes de hacer las pruebas vamos al controlador de login y hacemos unos cambios



```
1  // validamos el incio de sesion exitoso
2      res.status(200).json({
3          message:" Inicio de sesion correcto",
4          usuario:{
5              id: usuario._id,
6              nombre: usuario.Nombre,
7              apellido: usuario.Apellido,
8              email: usuario.Correo_Electronico,
9              Telefono: usuario.telefonotelefono
10         }
```

Llamamos al login. Js en nuestro html login

```
<script src="./java/login.js"></script>
```

Hacemos las pruebas y nos tiene que quedar bien el login

MODULO DE REGISTER



```
1 // LAMAMOS LA FUNCION PARA QUE TODO CARGUE
2 document.addEventListener('DOMContentLoaded', function() {
3
4     // SE REALIZA LA CONSTANTE DE REGISTER DE LA API
5
6     const API_URL = 'http://localhost:8081/api/users/register';
```

2.



```
1 // Mostrar/ocultar contraseñas
2 document.getElementById('toggle-password').addEventListener('click', function() {
3     const input = document.getElementById('password');
4     input.type = input.type === 'password' ? 'text' : 'password';
5 });
6
7 document.getElementById('toggle-confirm-password').addEventListener('click', function() {
8     const input = document.getElementById('confirmPassword');
9     input.type = input.type === 'password' ? 'text' : 'password';
10 });
```



```
1 // Enviar formulario
2 document.getElementById('register-form').addEventListener('submit', async function(e) {
3     e.preventDefault();
4
5     const errorDiv = document.getElementById('register-error');
6     const errorMsg = document.getElementById('register-error-message');
7     const btn = document.getElementById('register-btn');
8
9     // Obtener datos
10    const datos = {
11        Nombre: document.getElementById('firstName').value.trim(),
12        Apellido: document.getElementById('lastName').value.trim(),
13        Telefono: document.getElementById('phone').value.trim(),
14        Correo_Electronico: document.getElementById('email').value.trim(),
15        passwords: document.getElementById('password').value
16    };
17
```



```
1 const confirmPass = document.getElementById('confirmPassword').value;
2
3 // Validar contraseñas
4 if (datos.passwords !== confirmPass) {
5     errorMsg.textContent = 'Las contraseñas no coinciden';
6     errorDiv.classList.remove('hidden');
7     return;
8 }
9
10 // Validar términos
11 if (!document.getElementById('terms').checked) {
12     errorMsg.textContent = 'Debes aceptar los términos';
13     errorDiv.classList.remove('hidden');
14     return;
15 }
```



```
1 // Enviar al servidor
2   btn.disabled = true;
3   btn.textContent = 'Registrando...';
4
5   try {
6     const response = await fetch(`${API_URL}`, {
7       method: 'POST',
8       headers: { 'Content-Type': 'application/json' },
9       body: JSON.stringify(datos)
10    });
11
12    const result = await response.json();
13
```



```
1 if (response.ok) {
2   errorDiv.classList.remove('bg-red-50', 'border-red-200', 'text-red-800', 'hidden');
3   errorDiv.classList.add('bg-green-50', 'border-green-200', 'text-green-800');
4   errorMsg.textContent = '¡Registro exitoso! Redirigiendo...';
5   setTimeout(() => window.location.href = 'login.html', 7000);
6 } else {
7   errorMsg.textContent = result.message || 'Error al registrar';
8   errorDiv.classList.remove('hidden');
9   btn.disabled = false;
10  btn.textContent = 'Crear Cuenta';
11
12  } catch (error) {
13    errorMsg.textContent = 'Error de conexión';
14    errorDiv.classList.remove('hidden');
15    btn.disabled = false;
16    btn.textContent = 'Crear Cuenta';
17  }
18  });
19  });
```

Llamamos el js de register al html de register y hacemos las pruebas.

PERFIL

Creamos un controller que se llame Perfil.js

```
1
2 import Users from "../models/User.js";
3
4 // Obtener perfil del usuario
5 export const obtenerPerfil = async (req, res) => {
6   try {
7     const { email } = req.body;
8     if (!email) {
9       return res.status(400).json({ message: "Email es requerido" });
10    }
11
12    const usuario = await Users.findOne({ Correo_Electronico: email }).select('-passwords');
13
14    if (!usuario) {
15      return res.status(404).json({ message: "Usuario no encontrado" });
16    }
17  }
```

```
1   res.status(200).json({
2     usuario: {
3       id: usuario._id,
4       nombre: usuario.Nombre,
5       apellido: usuario.Apellido,
6       email: usuario.Correo_Electronico,
7       telefono: usuario.Telefono
8     }
9   });
10 } catch (error) {
11   res.status(500).json({
12     message: "Error al obtener perfil",
13     error: error.message
14   });
15 }
16 };
```

2. Creamos la ruta Perfil.js



```
1 // routes/perfil.routes.js
2 import express from "express";
3 import { obtenerPerfil, } from "../controllers/perfil.js";
4
5 const router = express.Router();
6
7 router.post('/obtener', obtenerPerfil);
8
9 export default router;
```

- Vamos a nuestro server y configuramos la nueva ruta



```
1 import PerfilRouter from './routes/Perfil.js'
```



```
1 app.use("/api/perfil", PerfilRouter);
```

2. En productos agregamos la siguientes línea



```
1 <!-- Cart and Mobile Menu MEJORADOS -->
2     <div class="flex items-center space-x-4">
3         <div id="user-menu-container"></div>
```

3. Ahora creamos el js que nos guardara el perfil en productos



```
1 document.addEventListener("DOMContentLoaded", async () => {
2
3     const sesionActiva = localStorage.getItem("sesionActiva");
4     const contenedor = document.getElementById("user-menu-container");
5
6     // Si no existe contenedor → está en otra página, no hacemos nada
7     if (!contenedor) return;
8
9     // Si no hay sesión → no mostrar menú
10    if (!sesionActiva) return;
```




```
1 // ☆ 1. TRAER DATOS DESDE LA API
2
3 const perfil = JSON.parse(localStorage.getItem("usuario"));
4 if (!perfil || !perfil.email) return;
5
6 let usuario = null;
7
8 try {
9     const res = await fetch("http://localhost:8081/api/perfil/obtener", {
10         method: "POST",
11         headers: { "Content-Type": "application/json" },
12         body: JSON.stringify({ email: perfil.email })
13     });
14
15     const data = await res.json();
16
17     if (!res.ok) throw new Error("No se pudo obtener perfil");
```



```
1 usuario = data.usuario;
2
3 } catch (err) {
4     console.error("Error al obtener usuario:", err);
5     // cerrar sesión si falla
6     localStorage.clear();
7     window.location.href = "../pages/login.html";
8     return;
9 }
```

```

1
2 // ☆ 2. CREAR MENÚ DEL USUARIO
3
4 contenedor.innerHTML = `
5   <div class="relative">
6     <button id="user-menu-btn"
7       class="w-14 h-14 rounded-full bg-blue-600 text-white flex items-center justify-center font-bold text-xl shadow-md hover:scale-105 transition-transform">
8       <span id="user-avatar"></span>
9     </button>
10
11     <div id="user-dropdown"
12       class="hidden absolute right-0 mt-2 w-60 bg-white rounded-xl shadow-2xl border border-gray-100 py-2 z-50
13       transition-all duration-200 ease-out overflow-hidden transform origin-top scale-95 opacity-0">
14
15       <div class="px-4 py-3 border-b border-gray-200">
16         <p class="text-sm font-semibold text-gray-900" id="user-name"></p>
17         <p class="text-xs text-gray-500" id="user-email"></p>
18       </div>
19
20       <a href="../pages/perfil.html"
21         class="flex items-center px-4 py-3 text-sm text-gray-700
22         hover:bg-blue-100 hover:text-blue-800
23         active:bg-blue-200 transition-all duration-150 rounded-md cursor-pointer">
24         Mi Perfil
25       </a>
26
27       <button id="logout-btn"
28         class="flex items-center w-full px-4 py-3 text-sm text-gray-600
29         hover:bg-blue-100 hover:text-blue-800
30         active:bg-blue-200 transition-all duration-150 rounded-md cursor-pointer">
31         Cerrar sesión
32       </button>
33     </div>
34   </div>
35 `;

```

```

1 // ☆ 3. INSERTAR DATOS EN EL MENÚ
2
3 document.getElementById("user-name").textContent =
4   `${usuario.nombre} ${usuario.apellido}`;
5
6 document.getElementById("user-email").textContent = usuario.email;
7
8 const avatar = `${usuario.nombre[0]}${usuario.apellido[0]}`.toUpperCase();
9 document.getElementById("user-avatar").textContent = avatar;

```

```
1 // ☆ 4. ANIMACIÓN ABRIR/CERRAR
2
3 document.getElementById("user-menu-btn").addEventListener("click", () => {
4     const drop = document.getElementById("user-dropdown");
5
6     if (drop.classList.contains("hidden")) {
7         drop.classList.remove("hidden");
8
9         setTimeout(() => {
10             drop.classList.remove("opacity-0", "scale-95");
11             drop.classList.add("opacity-100", "scale-100");
12         }, 20);
13
14     } else {
15         drop.classList.remove("opacity-100", "scale-100");
16         drop.classList.add("opacity-0", "scale-95");
17
18         setTimeout(() => {
19             drop.classList.add("hidden");
20         }, 150);
21     }
22 });
23
24 });
```

LLAMAMOS A JS EN PRODUCTOS Y HACEMOS LAS PRUEBAS Y HACEMOS LAS PRUEBAS Y LISTO YA ESTÁ NUESTRO PERFIL

ACTUALIZAR EL PERFIL

CONTROLLER PARA ACTUALIZAR EL PERFIL

```
1 // Actualizar perfil del usuario
2 export const actualizarPerfil = async (req, res) => {
3   try {
4     const { email, nombre, apellido, telefono } = req.body;
5
6     // Validar campos obligatorios
7     if (!email) {
8       return res.status(400).json({ message: "Email es requerido" });
9     }
10
11     if (!nombre || !apellido || !telefono) {
12       return res.status(400).json({ message: "Todos los campos son obligatorios" });
13     }
14   }
15 }
```

```
1 // Buscar y actualizar usuario
2 const usuarioActualizado = await Users.findOneAndUpdate(
3   { Correo_Electronico: email },
4   {
5     Nombre: nombre,
6     Apellido: apellido,
7     Telefono: telefono
8   },
9   { new: true }
10   // no va seleccionar el campo passwords
11 ).select('-passwords');
12
13 if (!usuarioActualizado) {
14   return res.status(404).json({ message: "Usuario no encontrado" });
15 }
```



```
1 res.status(200).json({
2     message: "Perfil actualizado exitosamente",
3     usuario: {
4         id: usuarioActualizado._id,
5         nombre: usuarioActualizado.Nombre,
6         apellido: usuarioActualizado.Apellido,
7         email: usuarioActualizado.Correo_Electronico,
8         telefono: usuarioActualizado.Telefono
9     }
10 });
11 } catch (error) {
12     res.status(500).json({
13         message: "Error al actualizar perfil",
14         error: error.message
15     });
16 }
17 };
```

Configúranos la rutas



```
1 import { obtenerPerfil, actualizarPerfil
```



```
1 router.put('/actualizar', actualizarPerfil);
```

Hacemos las pruebas en potman

1. Voy a traer un usuario

Vamos a Potman y validamos el usuario con el correo jalmpa77@gmail.com

POST

⌵

http://localhost:8081/api/Perfil/obtener

Params

Authorization

Headers (9)

Body ●

Scripts

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON ⌵

1 {

2 "email": "jalmpa77@gmail.com"

3 }

4

Para validar que si está funcionando el backend de actualizar

```
1  {
2    "usuario": {
3      "id": "690b820e55b1dd37ca330557",
4      "nombre": "Diego Fabian ",
5      "apellido": "Lombana Rivera",
6      "email": "jalmpa77@gmail.com",
7      "telefono": 3202358243
8    }
9  }
```

A ese usuario le vamos hacer el cambio



```
1  {
2    "email": "usuario@ejemplo.com",
3    "nombre": "Juan",
4    "apellido": "Pérez",
5    "telefono": "3001234567"
6  }
```

Y todo saldrá excelente

Controller de eliminar



```
1 export const eliminarPerfil = async (req, res) => {
2   try {
3     const { email } = req.body;
4
5     // Validar que el email esté presente
6     if (!email) {
7       return res.status(400).json({ message: "Email es requerido" });
8     }
9   }
10 }
```




```
1  // Buscar y eliminar usuario
2      const usuarioEliminado = await Users.findOneAndDelete({
3          Correo_Electronico: email
4      });
5
6      if (!usuarioEliminado) {
7          return res.status(404).json({ message: "Usuario no encontrado" });
8      }
9
10     res.status(200).json({
11         message: "Usuario eliminado exitosamente",
12         usuario: {
13             id: usuarioEliminado._id,
14             nombre: usuarioEliminado.Nombre,
15             apellido: usuarioEliminado.Apellido,
16             email: usuarioEliminado.Correo_Electronico
17         }
18     });
19 } catch (error) {
20     res.status(500).json({
21         message: "Error al eliminar perfil",
22         error: error.message
23     });
24 }
25 };
```

Configuramos rutas

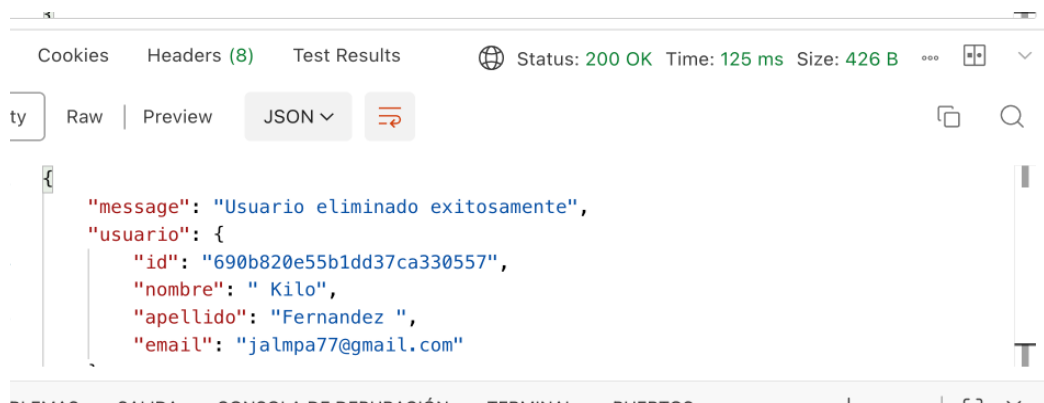
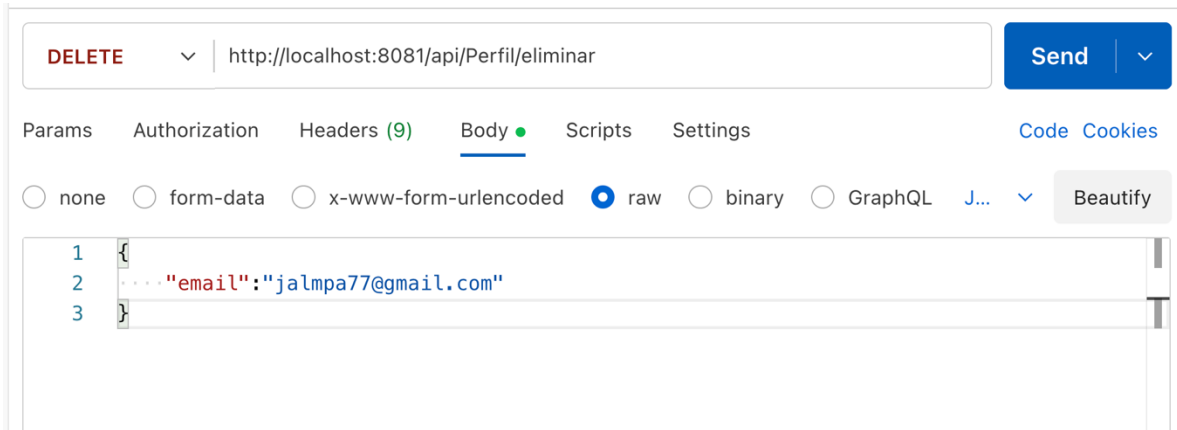
```
// routes/perfil.routes.js
import express from "express";
import { obtenerPerfil, actualizarPerfil, eliminarPerfil } from "../controllers";

const router = express.Router();

router.post('/obtener', obtenerPerfil);
router.put('/actualizar', actualizarPerfil);
router.delete('/eliminar', eliminarPerfil);

export default router;
```

Hacemos las pruebas en potman



Ustedes van a conectar el backend con el fronted

MODULO DE RECUPERAR CONTRASENA

1. INSTALAMOS UNA LIBRERÍA QUE SE LLAMA Nodemailer QUE ES EL ME PERMITE ENVIAR CORREOS DE FORMA SEGURA Y EFICAS

```
npm install nodemailer
```

2. CONTROLLERS , IMPORTAMOS LAS LIBRERIAS QUE SON NECESARIAS PARA CONTRUIR LA LOGICA DE RECUPERAR CONTRASENA.

JS recuperar.js

```
1  import bcrypt from "bcrypt";
2  import nodemailer from "nodemailer";
3  import Users from "../models/Users.js";
4
5  const transporter = nodemailer.createTransport({
6    service: 'gmail',
7    auth: {
8      user: 'jalmpa77@gmail.com',
9      pass: 'zkcqgiqsizkohbqt'
10   }
11 });
```

2. VAMOS A CREAR LA CONTRASEÑA APP DE GMAIL PARA PODER CONECTAR NUESTRO SERVICIO DE CORREO GMAIL
3. VAMOS A CREAR LA FUNCION QUE ME PERMITE SOLICITAR CODIGO DE VERIFICACION POR SEGURIDAD



```
1 // FUNCIÓN PARA GENERAR CÓDIGO DE 6 DÍGITOS
2 const generarCodigo = () => {
3     return Math.floor(100000 + Math.random() * 900000).toString();
4 };
```

SOLICITAR CODIGO DE VERIFICACION



```
1 // 1. SOLICITAR CÓDIGO DE RECUPERACIÓN
2 export const solicitarCodigo = async (req, res) => {
3     try {
4         const { Correo_Electronico } = req.body;
5
6         if (!Correo_Electronico) {
7             return res.status(400).json({
8                 message: "El correo electrónico es obligatorio"
9             });
10        }
11        // Buscar usuario
12        const usuario = await Users.findOne({ Correo_Electronico });
13
14        if (!usuario) {
15            return res.status(400).json({
16                message: "Correo electronico no encontrado "
17            });
18        }
19    }
20 };
```

GENERAR CODIGO DE 6 DIJITOS



```
1 // Generar código de 6 dígitos
2     const codigo = generarCodigo();
3
4     // Guardar código con expiración de 15 minutos
5     usuario.codigoRecuperacion = codigo;
6     usuario.codigoExpiracion = Date.now() + 900000; // 15 minutos
7     await usuario.save();
```

CONFIGURAR EL GMAIL DE RECEPCION

```

1  const mailOptions = {
2      from: 'jalmpa77@gmail.com',
3      to: usuario.Correo_Electronico,
4      subject: 'Código de Recuperación - TechStore Pro',
5      html: `
6          <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 20px;">
7              <div style="text-align: center; margin-bottom: 30px;">
8                  <h2 style="color: #4F46E5; margin: 0;">TechStore Pro</h2>
9              </div>
10
11              <h3 style="color: #333;">Recuperación de Contraseña</h3>
12
13              <p>Hola <strong>${usuario.Nombre}</strong>,</p>
14
15              <p>Recibimos una solicitud para restablecer tu contraseña.</p>
16
17              <p>Tu código de verificación es:</p>
18
19              <div style="background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
20                  padding: 20px;
21                  border-radius: 10px;
22                  text-align: center;
23                  margin: 30px 0;">
24                  <h1 style="color: black;
25                      font-size: 36px;
26                      letter-spacing: 8px;
27                      margin: 0;
28                      font-family: monospace;">
29                      ${codigo}
30                  </h1>
31              </div>
32
33              <p style="color: #666; font-size: 14px;">
34                  ⌚ Este código expirará en <strong>15 minutos</strong>.
35              </p>
36
37              <p style="color: #666; font-size: 14px;">
38                  🛡️ Si no solicitaste este cambio, ignora este email y tu contraseña permanecerá segura.
39              </p>
40
41              <hr style="margin: 30px 0; border: none; border-top: 1px solid #ddd;">
42
43              <p style="color: #999; font-size: 12px; text-align: center;">
44                  © 2025 TechStore Pro - Tu tienda de tecnología de confianza
45              </p>
46          </div>
47      `
48  };

```

ENVIAR EL GMAIL

```
1 // Enviar email
2     await transporter.sendMail(mailOptions);
3
4     console.log(`Código enviado a ${usuario.Correo_Electronico}: ${codigo}`);
5
6     res.status(200).json({
7         message: "Si el correo existe, recibirás un código de verificación",
8     });
9
10
11 } catch (error) {
12     console.error("Error al enviar código:", error);
13     res.status(500).json({
14         message: "Error al procesar la solicitud",
15         error: error.message
16     });
17 }
18 };
```

VERIFICAR CODIGO Y CAMBIAR CONTRASEÑA

```
1 // 2. VERIFICAR CÓDIGO Y CAMBIAR CONTRASEÑA
2 export const cambiarPassword = async (req, res) => {
3     try {
4         const { Correo_Electronico, codigo, nuevaPassword } = req.body;
5
6         // Validaciones
7         if (!Correo_Electronico || !codigo || !nuevaPassword) {
8             return res.status(400).json({
9                 message: "Todos los campos son obligatorios"
10            });
11        }
12
13        if (nuevaPassword.length < 6) {
14            return res.status(400).json({
15                message: "La contraseña debe tener al menos 6 caracteres"
16            });
17        }
18    }
19 };
```

VERIFICAR CODIGO Y CONTRASEÑA

```
1 // 2. VERIFICAR CÓDIGO Y CAMBIAR CONTRASEÑA
2 export const cambiarPassword = async (req, res) => {
3   try {
4     const { Correo_Electronico, codigo, nuevaPassword } = req.body;
5
6     // Validaciones
7     if (!Correo_Electronico || !codigo || !nuevaPassword) {
8       return res.status(400).json({
9         message: "Todos los campos son obligatorios"
10      });
11    }
12
13    if (nuevaPassword.length < 6) {
14      return res.status(400).json({
15        message: "La contraseña debe tener al menos 6 caracteres"
16      });
17    }
18  }
19 }
```

ENCRIPTAMOS LA CONTRASEÑA

```
1 // Encriptar nueva contraseña
2 const salt = await bcrypt.genSalt(10);
3 const hashedPassword = await bcrypt.hash(nuevaPassword, salt);
4
5 // Actualizar contraseña y limpiar código
6 usuario.passwords = hashedPassword;
7 usuario.codigoRecuperacion = undefined;
8 usuario.codigoExpiracion = undefined;
9 await usuario.save();
```


EMAIL DE CONFIRMACION

```
1  // Email de confirmación
2  const mailOptions = {
3    from: 'jalmpa77@gmail.com',
4    to: usuario.Correo_Electronico,
5    subject: 'Contraseña Actualizada - TechStore Pro',
6    html: `
7      <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 20px;">
8        <div style="text-align: center; margin-bottom: 30px;">
9          <div style="background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
10            width: 60px;
11            height: 60px;
12            border-radius: 50%;
13            display: inline-flex;
14            align-items: center;
15            justify-content: center;
16            margin-bottom: 20px;">
17            <span style="color: white; font-size: 30px;">✓</span>
18          </div>
19          <h2 style="color: #4F46E5; margin: 0;">Contraseña Actualizada</h2>
20        </div>
21
22        <p>Hola <strong>${usuario.Nombre}</strong>, </p>
23
24        <p>Tu contraseña ha sido actualizada exitosamente.</p>
25
26        <p>Ya puedes iniciar sesión con tu nueva contraseña.</p>
27
28        <div style="text-align: center; margin: 30px 0;">
29          <a href="http://127.0.0.1:5500/src/pages/login.html"
30            style="background: linear-gradient(to right, #4F46E5, #7C3AED);
31              color: white;
32              padding: 12px 30px;
33              text-decoration: none;
34              border-radius: 8px;
35              display: inline-block;">
36            Iniciar Sesión
37          </a>
38        </div>
39
40        <p style="color: #dc2626; font-size: 14px;">
41          ⚠ Si no realizaste este cambio, contacta a soporte inmediatamente.
42        </p>
43
44        <hr style="margin: 30px 0; border: none; border-top: 1px solid #ddd;">
45
46        <p style="color: #999; font-size: 12px; text-align: center;">
47          © 2025 TechStore Pro - Tu tienda de tecnología de confianza
48        </p>
49      </div>
50    `;
51  };
```

ENVIAR MENSAJE DE CONTRASEÑA ACTUALIZADA



```
1  await transporter.sendMail(mailOptions);
2
3      res.status(200).json({
4          message: "Contraseña actualizada exitosamente"
5      });
6
7  } catch (error) {
8      console.error("Error al cambiar contraseña:", error);
9      res.status(500).json({
10         message: "Error al cambiar la contraseña",
11         error: error.message
12     });
13  }
14  };
```

4. CREAMOS LA RUTA



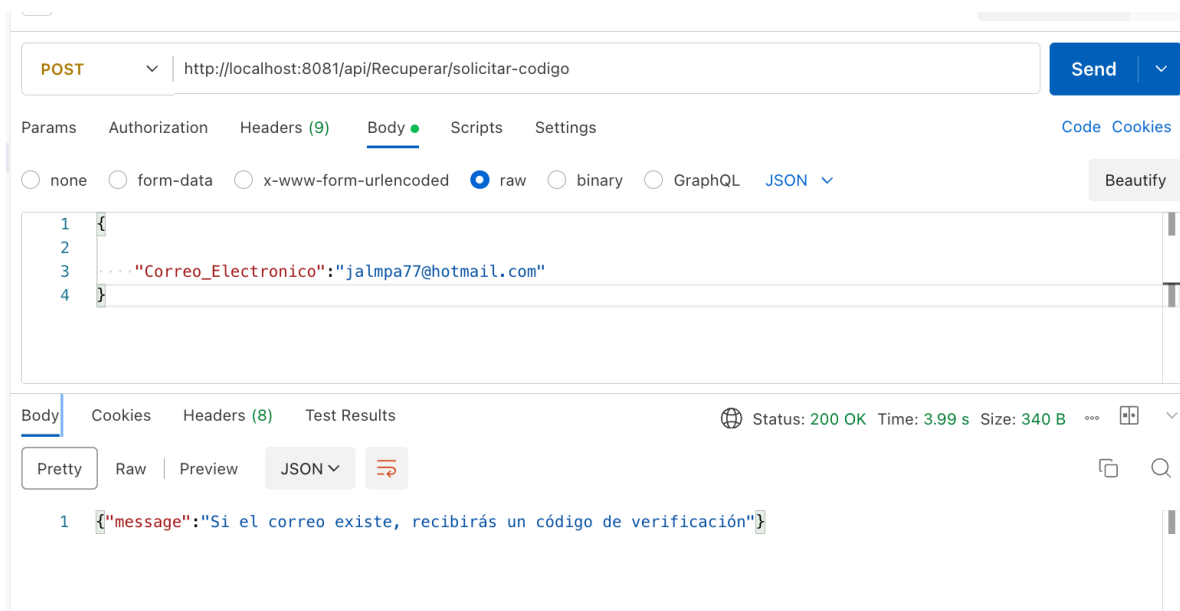
```
1  import express from "express";
2  import { solicitarCodigo, cambiarPassword } from "../controllers/recuperar.js";
3
4  const router = express.Router();
5
6
7  router.post('/solicitar-codigo', solicitarCodigo);
8
9
10 router.post('/cambiar-password', cambiarPassword);
11
12 export default router;
```

5. CONFIGURAMOS EL SERVER

```
import RecuperarPassword from './routes/recuperar.js'  
  
app.use('/api/Recuperar', RecuperarPassword)
```

REALIZAMOS LA PRUEBA EN POTMAN

1. Vamos a solicitar el código



VAMOS AL CORREO Y LLEGA EL CODIGO

Recuperación de Contraseña

Hola **JIMMY**,

Recibimos una solicitud para restablecer tu contraseña.

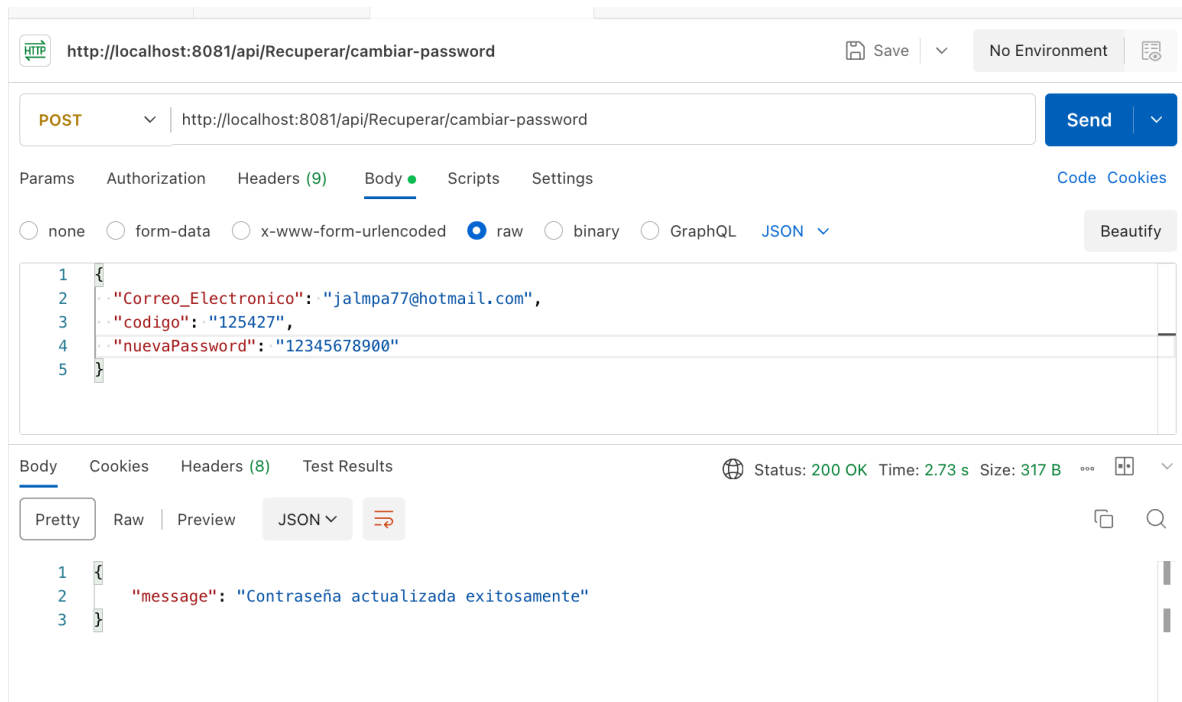
Tu código de verificación es:

1 2 5 4 2 7

 Este código expirará en **15 minutos**.

 Si no solicitaste este cambio, ignora este email y tu contraseña permanecerá segura.

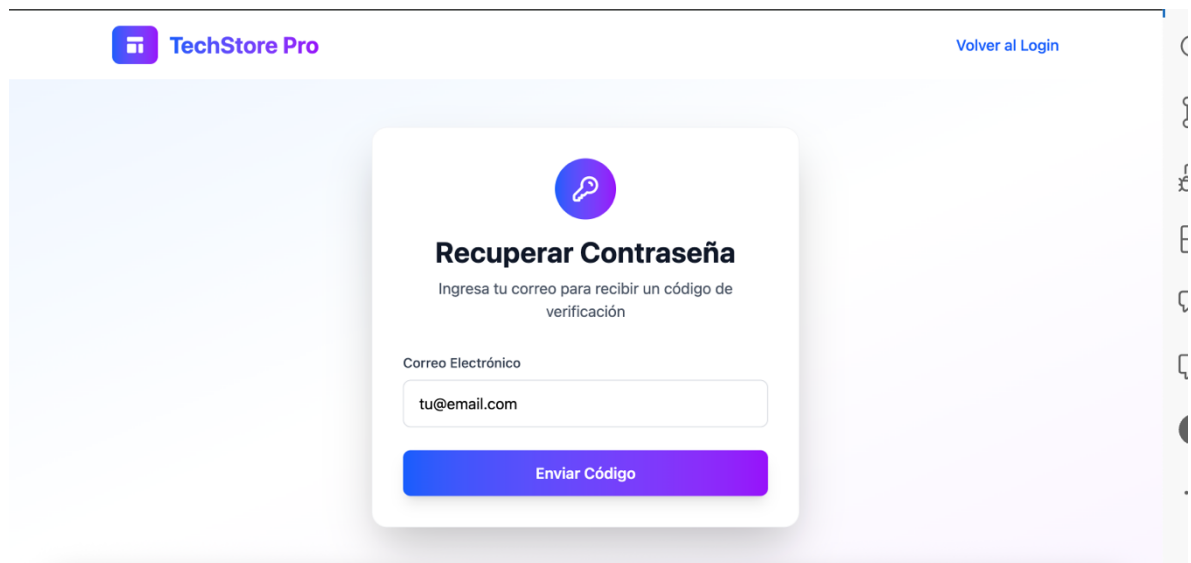
Ahora realizamos la prueba en cambiar contraseña



YA DE ESA MANERA PROBAMOS EL BACKEND

SEGUIDAMENTE SIGUE HACER EL FRONTED

1. PANTALLA EL CUAL ES DONDE VA ESCRIBIR EL CORREO



2. PANTALLA



Verificar Código

Ingresa el código enviado a jallmpa77@gmail.com

Código de Verificación

000000

Nueva Contraseña

.....

Confirmar Contraseña

.....

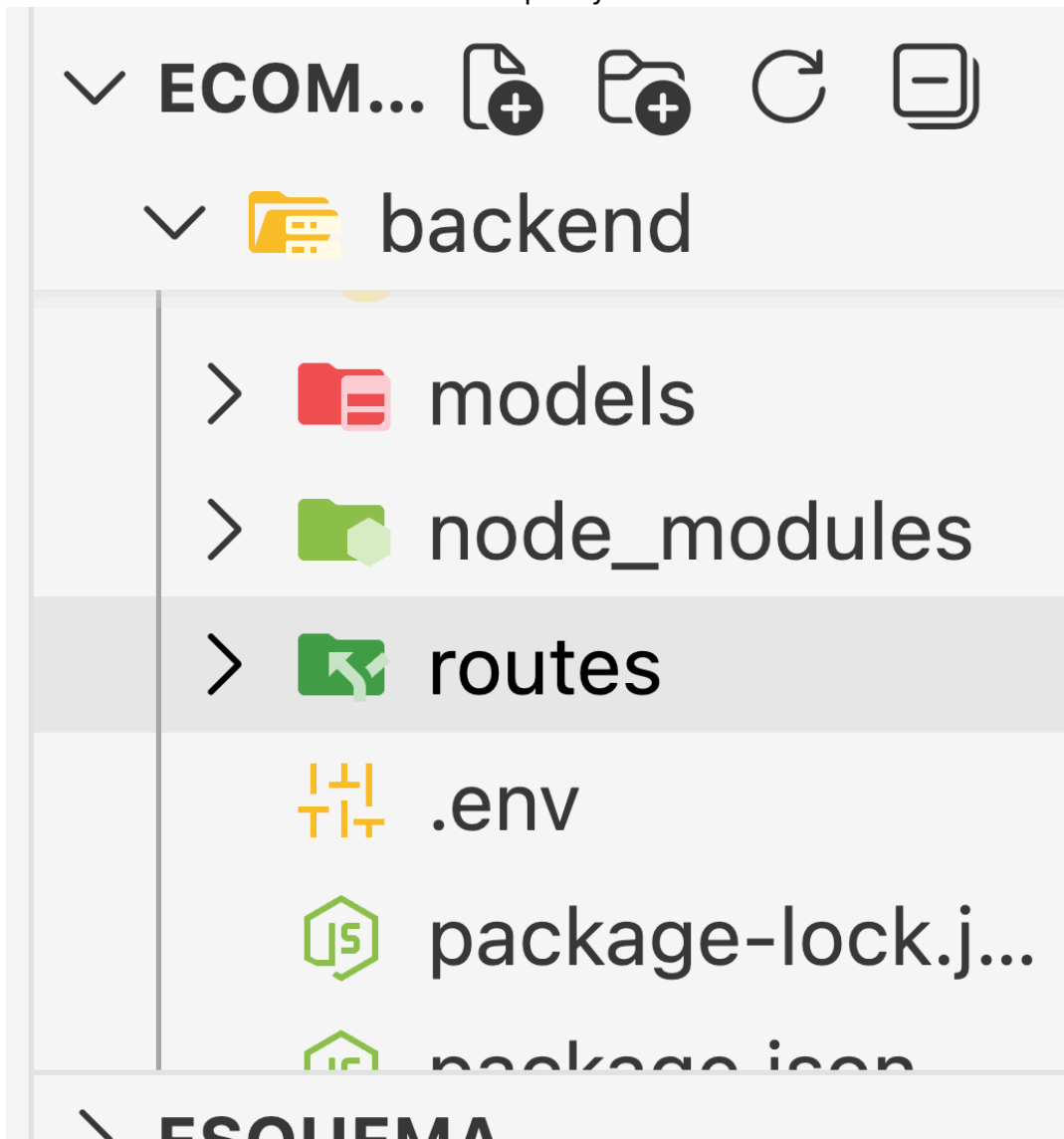
Cambiar Contraseña

ENTORNO DE VARIABLES

1. Abrimos el terminal

```
jimmyalexanderlombanarivera@MacBook-Pro-de-Jimmy backend %  
npm i dotenv
```

2. Vamos en la raíz de la Carpeta y creamos el archivo .env



3. Creamos las variables de entorno

Creamos la variable de la base de datos, del correo y email de la app de gmail

```
Import to Postman
MONGODB_URI=mongodb+srv://adsotarde:adsotarde@adsotarde.ech1xrh.mongodb.net/ECOMMER?retryWrites=true&w=majority
EMAIL_USER=jalmpa77@gmail.com
EMAIL_PASS=zkcggiqsizkohbqt
```

Ingresamos a la estructura de la base de datos y cambiamos por lo siguiente

```
import dotenv from "dotenv";

dotenv.config();

const uri = process.env.MONGODB_URI;
```

4. Controlador de recuperar

```
user: process.env.EMAIL_USER,
pass: process.env.EMAIL_PASS

import dotenv from "dotenv";

dotenv.config();
```


Hacemos las pruebas y debe quedar conectado la base de datos y el correo

MODULO DE CARRITO DE COMPRAS

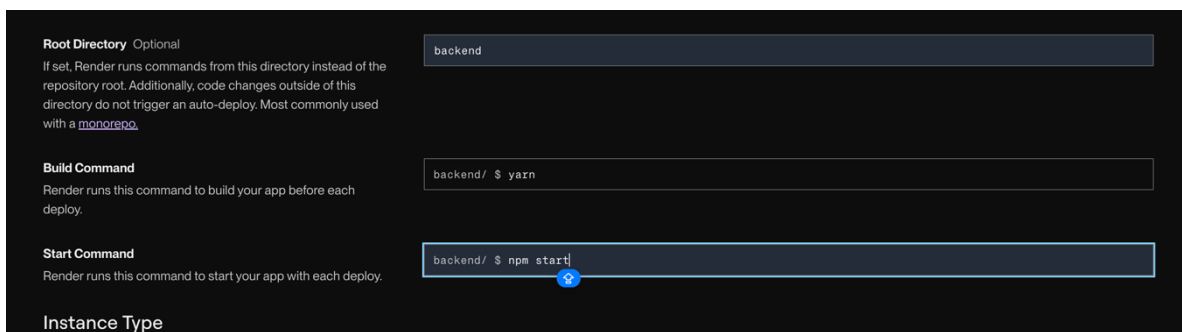
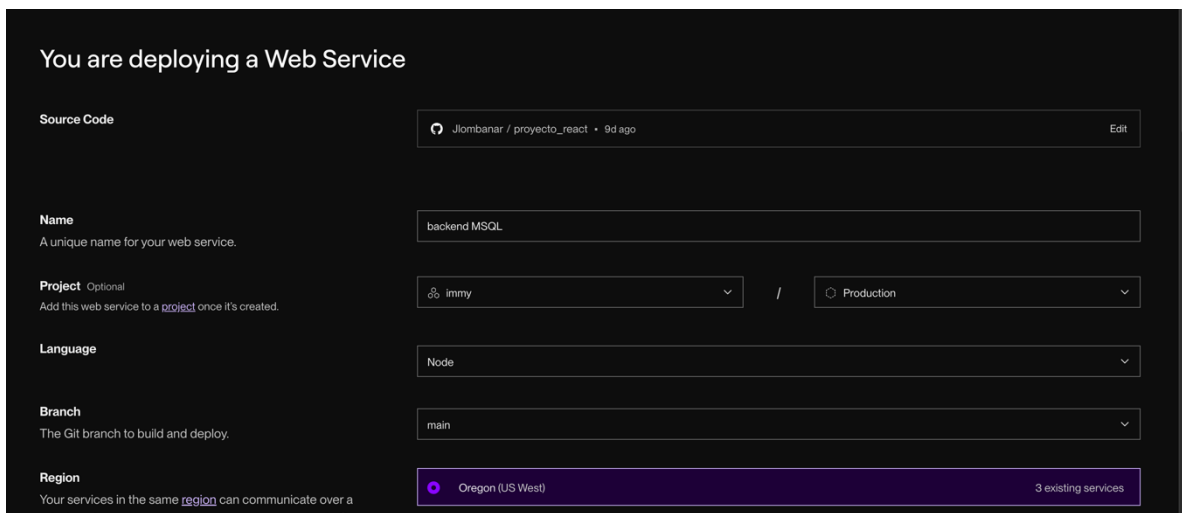
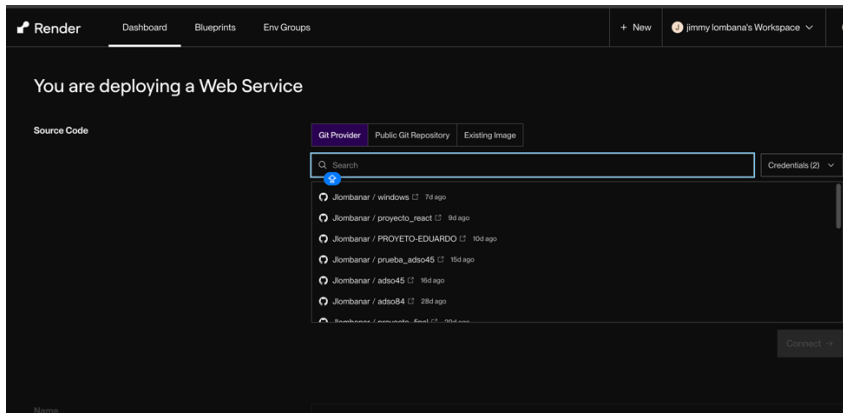
1. CREAMOS EL MODELO

```
1 import mongoose from 'mongoose';
2
3 const pedidoSchema = new mongoose.Schema({
4   userId: {type: String,required: true },
5   productos:[{productId: {type: String,required: true },
6     nombre: {type: String,required: true },
7     precio: {type: Number,required: true},
8     cantidad: {type: Number,required: true,default: 1 },
9     imagen: String}],
10   total: { type: Number,required: true},
11   estado: {type: String,enum: ['pendiente', 'procesando', 'enviado', 'entregado', 'cancelado'],default: 'pendiente' },
12   metodoPago: { type: String, required: true},
13   direccionEnvio: {calle: String, ciudad: String,codigoPostal: String,pais: String},
14   fechaPedido: {type: Date,default: Date.now}
15 });
16
17 export default mongoose.model('Pedido', pedidoSchema);
```

DESPLEGAR EL BACKEND

SUBIMOS AL REPOSITORIO EL PROYECTO

ABRIMOS RENDER Y SELECCIONAMOS NUEVO



October 16, 2024 at 9:29 AM Building

51937cc subiendo el backend Cancel deploy

All logs Q Search

Live tail GMT-5 ↑ ↗

Oct 16 09:29:48 AM ● ==> Cloning from https://github.com/Jlombanar/proyecto_react

Oct 16 09:29:49 AM ● ==> Checking out commit 51937cc541f48ad12494647df442d45183fccc35 in branch main

Oct 16 09:29:51 AM ● ==> Using Node.js version 20.15.1 (default)

Oct 16 09:29:51 AM ● ==> Docs on specifying a Node.js version: <https://render.com/docs/node-version>

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more](#)

NAME_OF_VARIABLE

value

Generate

+ Add Environment Variable

Add from .env

Advanced

Deploy Web Service

SUBIR NUESTRO FRONTED

1. Creamos un archivo llamado netlify.toml, dentro de este archivo colocamos

```

netlify.toml
1  [build]
2    command = "npm install && npm run build"
3    publish = "src"
4
5  [[redirects]]
6    from = "/*"
7    to = "/pages/index.html"
8    status = 200

```

Vamos a json y configuramos lo siguiente

```

    test: {
      "error": "Error: no test specified",
      "dev": "npx @tailwindcss/cli -i ./src/style/input.css -o ./src/style/output.css --watch",
      "build": "npx @tailwindcss/cli -i ./src/style/input.css -o ./src/style/output.css --minify"
    },
  },
  {
    "scripts": {
      "build": "tailwindcss -i ./src/style/input.css -o ./src/style/output.css --minify"
    }
  }
}

```

Corremos en el fornted la siguiente estructura

```
npm run build
```

Vamos a y creamos el nuevo proyecto en Netlify y seleccionamos gut hub

Branch to deploy

main



Base directory

The directory where Netlify installs dependencies and runs your build command.

Build command

npm install && npm run build

Examples: `jeekyll build`, `gulp build`, `make all`

Publish directory

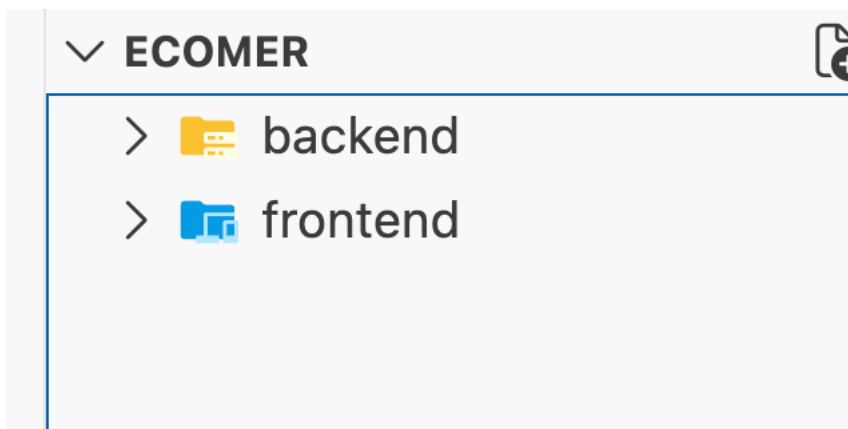
src

Examples: `_site`, `dist`, `public`

FRONTEND

CREAMOS UNA CARPETA Y PASAMOS EL BACKEND ABRIMOS CON VISUAL STUDIO CODE Y HACEMOS LAS PRUEBAS DEL BACKEND QUE SUBA SERVIDOR Y DE MAS .

CREAMOS UNA CARPETA DE FRONTEND



2. INGRESAMOS A LA CARPETA DEL DEL FRONTEND Y COMENZAMOS A REALIZAR LA INSTALACION DE REACT VITE

```
# 2. Crea el proyecto (te preguntará si quieres continuar, responde "y")  
npm create vite@latest . -- --template react  
  
# 3. Instala las dependencias  
npm install  
  
# 4. Inicia el servidor  
npm run dev
```



ES7 React/Redux/GraphQL

rodrigoallades | 3.107.055 | ★★★★★

Simple extensions for React, Redux and Grap...

Deshabilitar ▼

Desinstalar ▼

✓ Actualizaci

Actualizamos la estructura de nuestro app

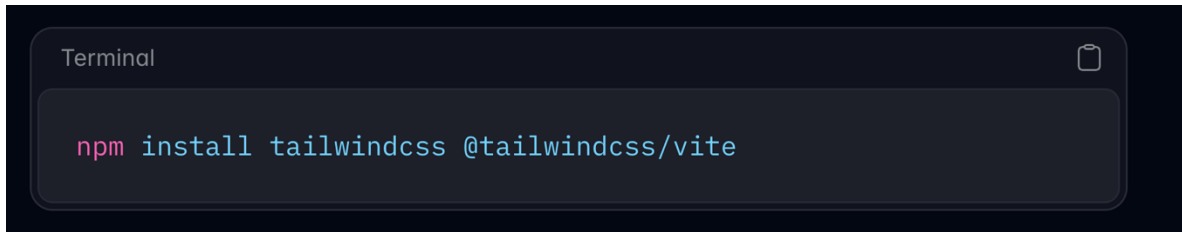
```
import './App.css'

function App() {

  return (
    <>
    <h1>Hola mundo </h1>
    </>
  )
}

export default App
```

INSTALACION DE TAIWIND EN REACT



```
Terminal

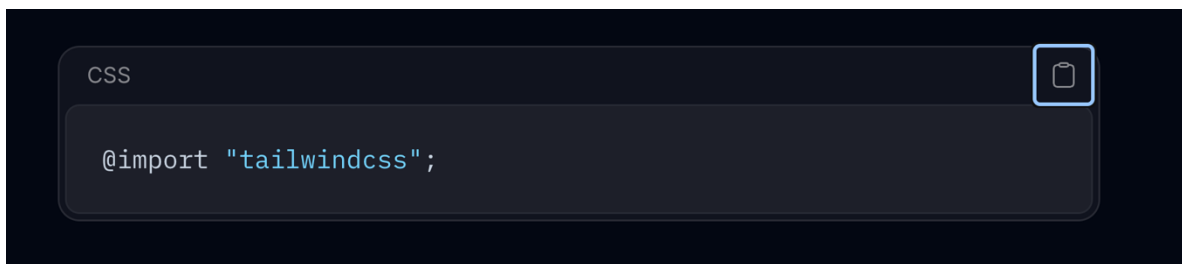
npm install tailwindcss @tailwindcss/vite
```

- configuramos vite, config y agregamos la siguiente ruta

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(),tailwindcss()],
})
```

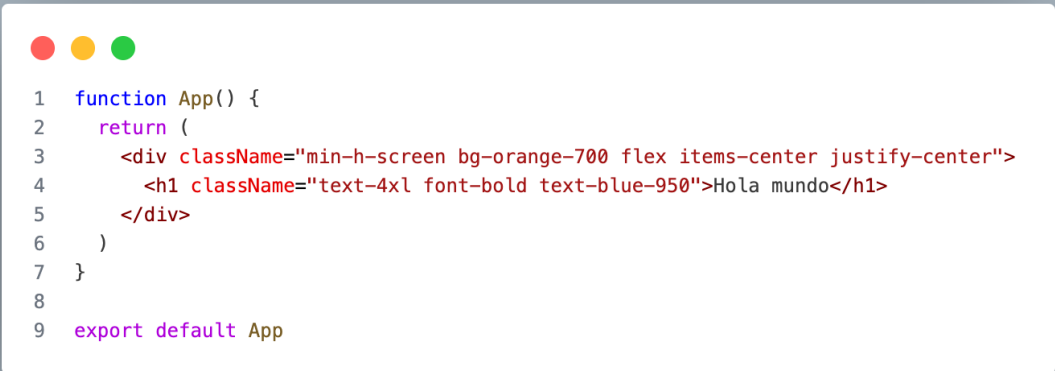
Vamos a index.css que trae por defecto copiamos lo siguientes



```
CSS

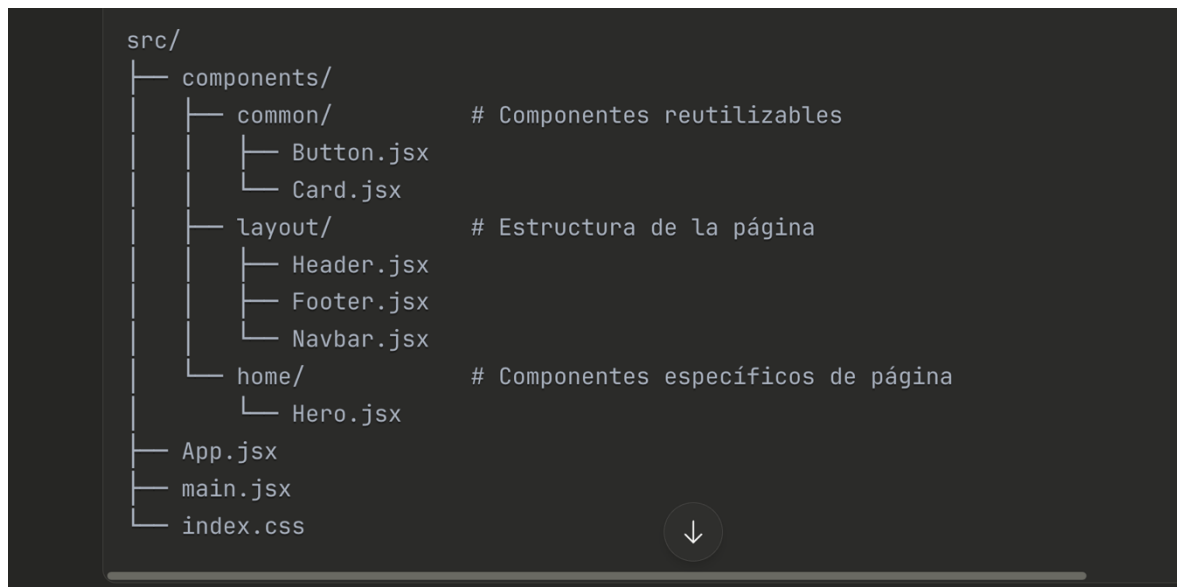
@import "tailwindcss";
```

Vamos a app.js y hacemos las pruebas que taiwind quedo instalado



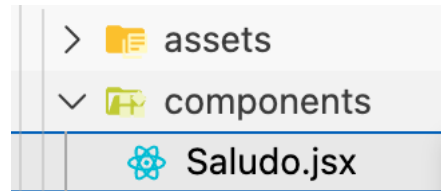
```
1 function App() {  
2   return (  
3     <div className="min-h-screen bg-orange-700 flex items-center justify-center">  
4       <h1 className="text-4xl font-bold text-blue-950">Hola mundo</h1>  
5     </div>  
6   )  
7 }  
8  
9 export default App
```

3. CREANDO NUESTRO PRIMER COMPONENTE



Clave principal todos los componentes de react deben empezar con mayúsculas

1. Creamos un componente llamado Saludo.jsx

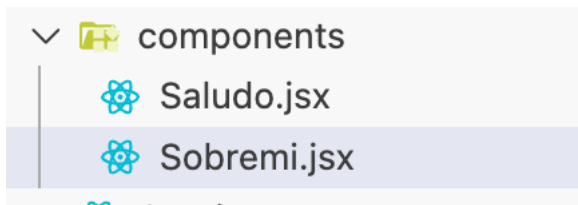


```
1 import jimmy from '../assets/jimmy.avif';
2
3 function Saludo() {
4   return (
5     <div className=
6       "min-h-screen w-full flex flex-col items-cent
7       er justify-center bg-amber-300 text-amber-950
8       p-6"
9     >
10       <img src={jimmy} alt="jimmy" className=
11         "w-40 h-40 rounded-full object-cover mb-4" />
12       <h2 className="text-2xl font-bold">
13         Hola mi nombre es Jimmy Lombana
14       </h2>
15       <p>
16         Estoy aprendiendo react a crear componets</p>
17     </div>
18   );
19 }
20
21 export default Saludo;
```

Vamos a app y hacemos lo siguiente sin el div

```
1 import Saludo from "../components/Saludo"
2 function App() {
3   return (
4     <div className="min-h-screen bg-amber-200 flex items-center">
5
6       <Saludo />
7
8     </div>
9   )
10 }
11
12 export default App
```

- Creamos otro componente sobre mi



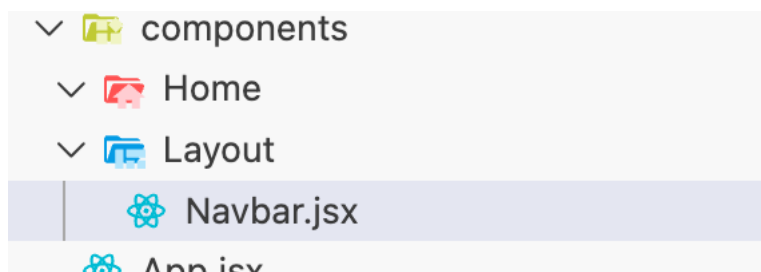
```
1 function Sobremi() {
2   return (
3     <div className="bg-blue-500 text-white p-6 rounded-lg max-w-md mx-auto text-center">
4       <h2 className="text-2xl font-bold">mi nombre es Jimmy Lombsns </h2>
5       <p className="mt-2">Estoy aprendiendo react con vite </p>
6     </div>
7   )
8 }
9
10 export default Sobremi
```

Vamos a app y lo llamamos

```
<Sobremi />
```

Vamos a empezar a trabajar ya nuestra estructura de nuestra pagina

- Vamos a crear nuestro componente Navbar dentro de la carpeta componentes vamos a crear una que se llame los Layout



Abrimos nuestro terminal y vamos a instalar una librería de iconos que trae react

```
bash
```

```
npm install lucide-react
```



```
1 import { useState } from 'react';  
2 import { ShoppingCart, User, Menu, X } from 'lucide-react';
```



```

1  /* Desktop Menu */
2  <div className="hidden md:flex space-x-6">
3    <a href="#inicio" className="text-gray-700 hover:text-blue-600 font-medium transition-colors duration-200 relative group">
4      Inicio
5      <span className="absolute -bottom-1 left-0 w-full h-0.5 bg-blue-600"></span>
6    </a>
7    <a href="#productos" className="text-gray-700 hover:text-blue-600 font-medium transition-colors duration-200 relative group">
8      Productos
9      <span className="absolute -bottom-1 left-0 w-0 h-0.5 bg-blue-600 transition-all duration-200 group-hover:w-full"></span>
10   </a>
11   <a href="#categorias" className="text-gray-700 hover:text-blue-600 font-medium transition-colors duration-200 relative group">
12     Categorías
13     <span className="absolute -bottom-1 left-0 w-0 h-0.5 bg-blue-600 transition-all duration-200 group-hover:w-full"></span>
14   </a>
15   <a href="#contacto" className="text-gray-700 hover:text-blue-600 font-medium transition-colors duration-200 relative group">
16     Contacto
17     <span className="absolute -bottom-1 left-0 w-0 h-0.5 bg-blue-600 transition-all duration-200 group-hover:w-full"></span>
18   </a>
19 </div>

```

```

1  /* Cart and Login Icons */
2  <div className="flex items-center space-x-2">
3    /* Cart Icon */
4    <
5      href="#cartrito"
6      className="relative group p-2.5 hover:bg-linear-to-r hover:from-blue-50 hover:to-indigo-50 rounded-xl transition-all duration-300 transform hover:scale-105"
7    >
8      <ShoppingCart className="w-6 h-6 text-gray-700 group-hover:text-blue-600 transition-all duration-300 group-hover:rotate-3" />
9      {count} > 0 && (
10        <span className="absolute -top-2 -right-2 bg-linear-to-r from-red-500 via-pink-500 to-red-600 text-white text-xs font-bold rounded-full min-w-5 h-5 flex items-center justify-center px-1 shadow-lg border-2 border-white animate-pulse">
11          {cartCount}
12        </span>
13      )
14    </div>
15  </div>

```

```

1
2  { /* Login Icon */
3  <a
4    href="#login"
5    className="relative group p-2.5 hover:bg-linear-to-r hover:from-blue-50 hover:to-indigo-50 rounded-xl transition-all duration-300 transform hover:scale-105"
6  >
7    <User className="w-6 h-6 text-gray-700 group-hover:text-blue-600 transition-all duration-300" />
8  </a>
9
10 { /* Mobile Menu Button */
11 <button
12   onClick={() => setMobileMenuOpen(!mobileMenuOpen)}
13   className="md:hidden p-2.5 hover:bg-linear-to-r hover:from-blue-50 hover:to-indigo-50 rounded-xl transition-all duration-300"
14 >
15   {mobileMenuOpen ? (
16     <X className="w-6 h-6 text-gray-700" />
17   ) : (
18     <Menu className="w-6 h-6 text-gray-700" />
19   )}
20 </button>
21 </div>
22 </div>

```

```

1  { /* Mobile Menu */
2    {mobileMenuOpen && (
3      <div className="md:hidden mt-4 py-4 border-t border-gray-200 animate-in slide-in-from-top duration-300">
4        <div className="flex flex-col space-y-4">
5          <a href="#inicio" className="text-gray-700 hover:text-blue-600 font-medium transition-colors duration-200 py-2">
6            Inicio
7          </a>
8          <a href="#productos" className="text-gray-700 hover:text-blue-600 font-medium transition-colors duration-200 py-2">
9            Productos
10         </a>
11         <a href="#categorias" className="text-gray-700 hover:text-blue-600 font-medium transition-colors duration-200 py-2">
12           Categorías
13         </a>
14         <a href="#contacto" className="text-gray-700 hover:text-blue-600 font-medium transition-colors duration-200 py-2">
15           Contacto
16         </a>
17       </div>
18     </div>
19   )}
20 </nav>
21 </header>
22 );
23 }
24
25 export default Navbar;

```

Vamos al app y configuramos el componente

Creamos una carpeta llamada Home y ahí creamos los componentes siguientes



Y colocamos el siguientes código

```
1 function Hero() {
2   return (
3     <section className="bg-linear-to-r from-blue-600 via-purple-600 to-indigo-600 text-white py-20 relative overflow-hidden">
4       <div className="absolute inset-0 bg-black opacity-10"></div>
5       <div className="container mx-auto px-4 text-center relative z-10">
6         <h1 className="text-5xl md:text-6xl font-bold mb-6 animate-fade-in">
7           Encuentra la mejor <span className="text-yellow-300">tecnología</span>
8         </h1>
9         <p className="text-xl md:text-2xl mb-8 opacity-90">
10           Laptops, celulares y componentes PC al mejor precio del mercado
11         </p>
12         <div className="flex flex-col sm:flex-row gap-4 justify-center">
13           <a
14             href="#productos"
15             className="bg-white text-blue-600 px-8 py-4 rounded-lg text-lg font-semibold hover:bg-gray-100 transition duration-300 transform hover:scale-105"
16           >
17             Ver Productos
18           </a>
19           <button className="border-2 border-white text-white px-8 py-4 rounded-lg text-lg font-semibold hover:bg-white hover:text-blue-600 transition duration-300">
20             Ofertas Especiales
21           </button>
22         </div>
23       </div>
24     </section>
25   );
26 }
27
28 export default Hero;
```

Configuramos el app

Seguimos con el siguiente componente.

Categoría



```
1  function Categories() {  
2    const categories = [  
3      {  
4        id: 'laptops',  
5        emoji: '💻',  
6        title: 'Laptops',  
7        description: 'Las mejores marcas y modelos para trabajo y gaming'  
8      },  
9      {  
10       id: 'celulares',  
11       emoji: '📱',  
12       title: 'Celulares',  
13       description: 'Última tecnología móvil de todas las marcas'  
14     },  
15     {  
16       id: 'componentes',  
17       emoji: '⚙️',  
18       title: 'Componentes PC',  
19       description: 'Arma tu PC ideal con los mejores componentes'  
20     }  
21   ];  
22 }
```



```

1   return (
2     <section id="categorias" className="py-16 bg-gray-50">
3       <div className="container mx-auto px-4">
4         <div className="text-center mb-12">
5           <h2 className="text-4xl font-bold mb-4">Nuestras Categorías</h2>
6           <p className="text-gray-600 text-lg">Descubre nuestra amplia gama de productos tecnológicos</p>
7         </div>
8
9         <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8">
10          {categories.map((category) => {
11            <div
12              key={category.id}
13              onClick={() => handleCategoryClick(category.id)}
14              className="bg-white p-8 rounded-xl shadow-lg text-center hover:shadow-2xl transition duration-300 transform hover:-translate-y-2 cursor-pointer"
15            >
16              <div className="text-6xl mb-6 filter drop-shadow-lg">
17                {category.emoji}
18              </div>
19              <h3 className="text-2xl font-bold mb-4 text-gray-800">
20                {category.title}
21              </h3>
22              <p className="text-gray-600 mb-6">
23                {category.description}
24              </p>
25              <div className="text-blue-600 font-semibold">Ver más </div>
26            </div>
27          )}
28        </div>
29      </div>
30    </section>
31  );
32 }
33
34 export default Categories;

```

Configuramos el app